



沈阳建筑大学

上机实验报告

课程名称:	编译原理
实验题目:	实验一 词法分析
专业班级:	计算机 2101 班
姓 名:	张清晨
学 号:	2104230414
日 期:	2024.04.27

【实验目的】

通过设计、调试词法分析程序，实现从源程序中分出各种单词的方法；熟悉词法分析程序所用的工具自动机，进一步理解自动机理论。掌握文法转换成自动机的技术及有穷自动机实现的方法。确定词法分析器的输出形式及标识符与关键字的区分方法。加深对课堂教学的理解；提高词法分析方法的实践能力。

内容：

- (1) 掌握文法转换成自动机的技术及有穷自动机实现的方法；
- (2) 确定词法分析器的输出形式及标识符与关键字的区分方法；
- (3) 通过设计、编制、调试词法分析程序，实现从源程序中拆分出各种单词。

【基本原理和上机步骤】

基本原理：

词法分析程序的基本任务是从字符串表示的源程序中识别出具有独立意义的单词符号，包括：关键字、标识符、数字、运算符、界限符。在成功识别这些单词符号后，依次输出各个单词符号的 token 表示（单词类别，属性），单词类别用于区分单词所属的类别，以整数编码表示。属性用于区分该类别中哪一个单词符号，即单词符号的值。

标识符：用来命名程序中出现的变量、数组、函数、过程、标号等，通常是字母打头的字母数字串，如 length、nextch 等；

关键字：具有标识符的形式，但不是由用户定义的，而是由语言定义的，其意义是约定的，如 if、while、for、int、do 等；

常数：由数字构成，包括各种类型的常数，如整型、浮点型等；

运算符：算数运算符：+ - × ÷；关系运算符：< > ≤；逻辑运算符：&& ||

界限符：;(){}

上机步骤：

- 1) 获取 FA
 - a. 程序设计语言词法规则⇒正规文法⇒FA
 - b. 词法规则⇒正规表达式⇒ FA;
- 2) FA 确定化⇒ DFA;
- 3) DFA 最简化;
- 4) 确定单词符号输出形式;
- 5) 化简后的 DFA+单词符号输出形式⇒构造词法分析程序。

【实验结果】

1.准备测试数据：新建 example_1.txt 和 example_2.txt 文件，内容如下

example1_1.txt	example2_1.txt
<pre>main() { int a, b; a = 10; b = a + 20; }</pre>	<pre>while(i > 5) { i = i++; }</pre>

要求：

识别保留字：Begin、end、if、then、else、while、do；

单词种别码为 1、2、3、4、5、6、7。

符号：,、,;、{、}、(、)

单词种别码为：8、9、10、11、12、13

运算符包括：+、-、*、/、<=、<>、<、=、>、>=

单词种别码为 14、15、16、17、18, 19, 20, 21, 22, 23。

标识符种别码为 24、整常数种别码为 25.

词法分析程序所用的全局变量和需调用的函数如下：

- 1) ch 字符变量，存放当前读进的源程序字符。
- 2) fgetch() 读字符函数，每调用一次从输入文件中读进源程序的下一个字符放在 ch 中，并把读字符指针指向下一个字符。

2. 运行结果：

```

E:\编译原理\上机实验\实验1\证 x  +  E:\编译原理\上机实验\实验1\证 x
(1,main)
(5,(
(5,))
(5,{
(2,int)
(1,a)
(5,,)
(1,b)
(5,;)
(1,a)
(4,=)
(3,10)
(5,;)
(1,b)
(4,=)
(1,a)
(4,+)
(3,20)
(5,;)
(5,})

(1,while)
(5,(
(1,i)
(4,>)
(3,5)
(5,))
(5,{
(1,i)
(4,=)
(1,i)
(4,++)
(5,;)
(5,})

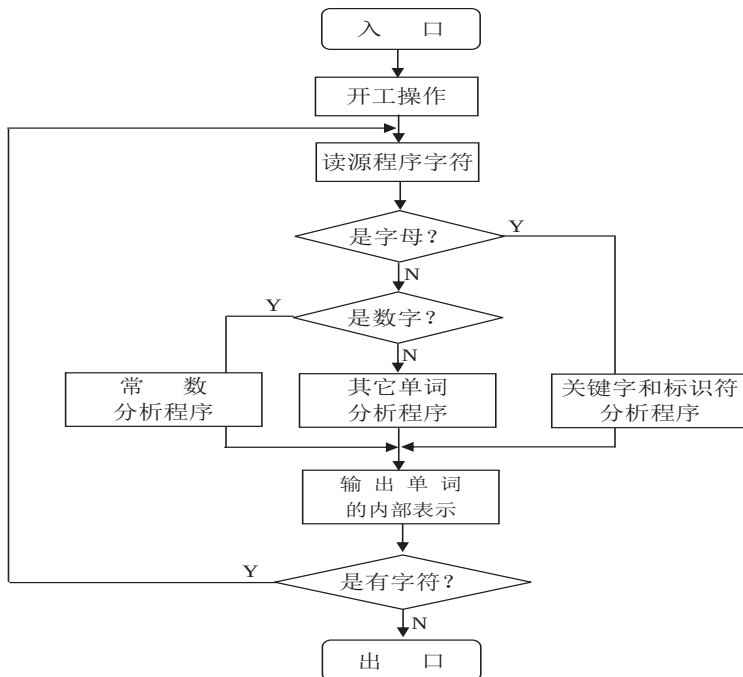
```

结果和预期的相符，程序运行正确，词法分析成功！

【讨论与分析】

词法分析程序的结果与预期的结果相同，这说明了可以根据不同符号的编码种类，预先画好状态转换图，只要构造出识别语言单词符号的有穷自动机，就很容易构造出识别语言单词符号的词法分析程序。词法分析程序的功能是从左到右扫描源程序字符串，根据语言的语法规则识别出各类单词符号。

对应本次的词法分析程序，代码的业务逻辑如下所示：



词法分析程序流程图

实验过程记录: 在实验的过程中,我遇到了几个小问题,分别是: 1. >= <= != 的实现问题 2. ++ -- 的问题。最开始不能检测到这几个符号,最后通过慢慢调试,查阅博客,最终得以实现。

实验总结: 在本次词法分析实验中,我不仅加深了对词法分析原理的理解,还提高了对正则表达式的应用能力。通过动手实现词法分析器,我对编译过程中的第一步有了更深入的认识,同时也为后续的语法分析和语义分析打下了坚实的基础。在实验过程中,我遇到了一些挑战,但通过独立调试和查找资料,我成功解决了这些问题。这些经验对我未来编译器设计和实现将具有极大的帮助。总的来说,这次实验让我受益匪浅。

【源程序代码】

```
#include <iostream>
#include <fstream>
#include <cassert>
#include <string>

using namespace std;

//判断当前字符串是否为关键字
bool isKey(string s)
{
    //关键字数组
    string keyArray[] = {"int", "char", "string", "bool", "float", "double", "true",
"false", "return",
                                "if", "else", "while", "for", "default", "do", "public",
"static", "switch"};
    //与当前字符串一一对比
    for (int i = 0; i < sizeof(keyArray); i++)
    {
        if (keyArray[i] == s)
            return true;
        else
            return false;
    }
}

//判断当前字符是否是运算符
bool isOperator(char ch)
```

```

{
    if ('+' == ch || '-' == ch || '*' == ch || '/' == ch || '%' == ch || '^' == ch || '=' == ch ||
'<' == ch || '>' == ch || '!' == ch)
        return true;
    else
        return false;
}

//判断当前字符是否是界限符分隔符
bool isSeparator(char ch)
{
    if( ',' == ch || ';' == ch || '{' == ch || '}' == ch || '(' == ch || ')' == ch)
        return true;
    else
        return false;
}

int main( )
{
    //定义字符变量，保存从源程序中读取的单个字符
    char ch;
    //定义字符串，保存从源程序中连续读取的字符串
    string result;
    //存放每个获取的单词的值
    string resultArray[999];
    //记录获取单词的个数
    int resultNum = 0;

    //代码存放的文件名
    string file = "example1_1.txt";
    //string file = "example2_1.txt";

    ifstream infile;
    //将文件流对象与文件连接起来
    infile.open(file.data());
    //若失败,则输出错误消息,并终止程序运行
    assert(infile.is_open());
}

```

```

//txt 文本中读取空格符与换行符
//infile >> noskipws;
//读取文本中的一个字符
infile >> ch;

while (!infile.eof())
{
    //ch 是英文字母或者下划线
    if(isalpha(ch) || '_'==ch)
    {
        result.append(1,ch);
        infile >> ch;
        //判断是否为关键字
        if(isKey(result))
        {
            resultArray[resultNum++] = "(2," + result + ")";
            result = "";
        }
        //读入首字符为字母，继续读入字母、数字、下划线，组成标识符
        或者关键字
        while(isalpha(ch) || isdigit(ch) || '_' == ch)
        {
            result.append(1,ch);
            infile >> ch;
            if(isKey(result))
            {
                resultArray[resultNum++] = "(2," + result + ")";
                result = "";
            }
        }
        //读入操作符或者运算符，正确保存标识符或者关键字
        if(isSeparator(ch) || isOperator(ch))
        {
            if(isKey(result))
            {
                resultArray[resultNum++] = "(2," + result + ")";
                result = "";
                continue;
            }
        }
    }
}

```

```

        else
        {
            if(isdigit(result.at(0)))
                resultArray[resultNum++] = "(Error," + result + ")";
            else
                resultArray[resultNum++] = "(1," + result + ")";
            result = "";
            continue;
        }

    }
    //读入不是字母、数字、运算符、标识符，继续读入直到遇到运算符或者分隔符
    else
    {
        result.append(1,ch);
        infile >> ch;
        while(!isSeparator(ch) && !isOperator(ch))
        {
            result.append(1,ch);
            infile >> ch;
        }
        resultArray[resultNum++] = "(Error," + result + ")";
        result = "";
        continue;
    }
}
//读入数字
else if(isdigit(ch))
{
    result.append(1,ch);
    infile >> ch;
    //继续读入数字，组成常数
    while(isdigit(ch))
    {
        result.append(1,ch);
        infile >> ch;
    }
    //遇到操作符或者运算符，正常终止

```



```

if(isOperator(ch) || isSeparator(ch))
{
    resultArray[resultNum++] = "(3," + result + ")";
    result="";
    continue;
}
//读入其他错误字符
else
{
    result.append(1,ch);
    infile >> ch;
    while(!isSeparator(ch) && !isOperator(ch))
    {
        result.append(1,ch);
        infile >> ch;
    }
    resultArray[resultNum++]="(Error,"+result+)";
    result = "";
    continue;
}
}
//遇到运算符
else if(isOperator(ch))
{
    result.append(1,ch);
    infile >> ch;
    //判断是否存在<=、>=、!=
    if("<" == result || ">" == result || "!=" == result)
    {
        if('='==ch)
        {
            result.append(1,ch);
            infile >> ch;
        }
    }
    //判断是否存在++
    if("+" == result)
    {
        if('+'==ch)

```

```

        {
            result.append(1,ch);
            infile >> ch;
        }
    }
    //判断是否存在--
    if("-" == result)
    {
        if('-'==ch)
        {
            result.append(1,ch);
            infile >> ch;
        }
    }
    //下一个读入符为字母、数字、分隔符，即正确
    if(isalpha(ch) || isdigit(ch) || isSeparator(ch))
    {
        resultArray[resultNum++] = "(4," + result + ")";
        result = "";
        continue;
    }
    else
    {
        //将错误输入符一起读入，直到正确
        while(!isSeparator(ch) && !isalpha(ch) && !isdigit(ch))
        {
            result.append(1,ch);
            infile >> ch;
        }
        resultArray[resultNum++] = "(Error,"+result+")";
        result = "";
        continue;
    }
}
//读取到分隔符
else if(isSeparator(ch))
{
    result.append(1,ch);
    resultArray[resultNum++] = "(5," + result + ")";
}

```

```

        result = "";
        infile >> ch;
    }
    //读取到未定义输入
    else{
        //出错处理
        result.append(1,ch);
        resultArray[resultNum++] = "(Error," + result + ")";
        result = "";
        infile >> ch;

    }
}
//关闭文件输入流
infile.close();

//以 （单词类编码， 值） 输出结果
for(int i=0;i<resultNum;i++){
    cout << resultArray[i] << endl;
}

return 0;
}

```